MDQ Specification Audit: ordering: mdq-spec

Date: 2026-10-21

Scope: Ordering question type: `docs/question-types/ordering.md`, its grammar in `docs/references/grammar.md`, `schema/ordering.yaml`, and the shared rules in `question-base.md`, `exam.md`, `responses.md` and `patterns.md` as they apply to ordering. Corpus examples under `examples/` for ordering.

Methodology

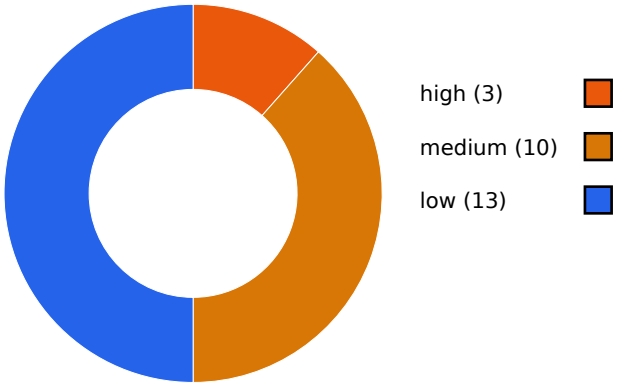
Scope: the ordering question type. Read docs/question-types/ordering.md, question-base.md, exam.md, responses.md, references/grammar.md and patterns.md, schema/ordering.yaml and the ordering corpus. scripts/spec_audit.py passed all 15 steps before the audit.

A grammar subagent turned the ordering snippet into Lark with stubs and tested what it accepts and rejects against the prose, the Additional Rules table and the corpus. A type scanner subagent reviewed prose, rules table, schema and corpus independently, and ran the complete examples through `mdq validate`. Edge cases were checked against mdq-py (`mdq validate`, `mdq show`, `mdq.parse`).

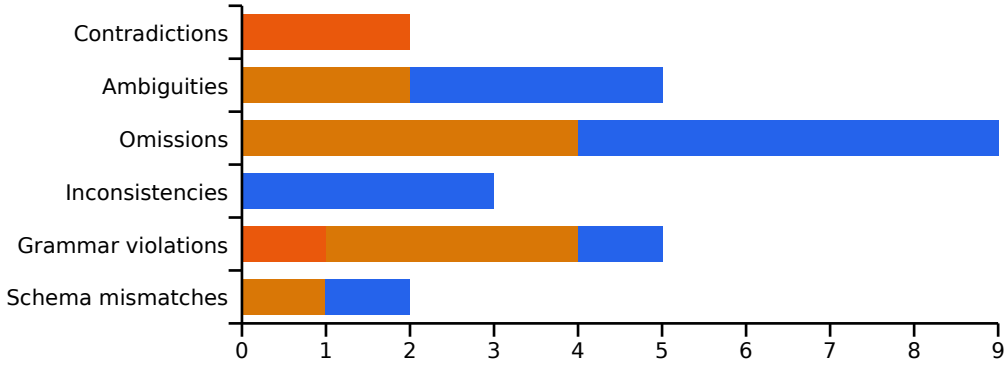
Executive summary

Severity	#
CRITICAL	0
HIGH	3
MEDIUM	10
LOW	13
INFO	0
Total findings	**26**

Findings by severity



Findings by category



Strengths and weaknesses

Strengths

✓ **Grammar violations:** Ordering grammar places sections strictly after the epilogue and rejects prose after a section. Tested with Lark (Earley) and stubs: epilogue then `## [extra]` ACCEPTED; prose after the last section REJECTED; prose between `## [extra]` and `## [reject]` REJECTED; examples/invalid/ordering-epilogue-after-section.mdq.md and ordering-prose-between-sections.mdq.md REJECTED; examples/valid/ordering/epilogue.mdq.md ACCEPTED. This matches the prose at ordering.md:175-182 and the Additional Rules row at line 340. (docs/question-types/ordering.md:141-148)

✓ **Grammar violations:** Ordering accept/reject sections with the shared observations rule enforce at most one feedback and one comment block, in either order, and no observations on [extra]. Tested: comment lines then blank then code ACCEPTED; > then ! with no blank ACCEPTED; feedback with a lazy line ACCEPTED; >/!/> interleaved REJECTED; two feedback blocks split by a blank line REJECTED; feedback under `## [extra]` REJECTED; heading with observations and no content REJECTED; examples/invalid/ordering-interleaved-observations and ordering-two-feedback-blocks REJECTED, examples/valid/ordering/observations-adjacent ACCEPTED. (docs/question-types/ordering.md:146-148)

✓ **Grammar violations:** Ordering tag and heading tokens hold for the stated literals: `[ordering]` with a code block or a `ul` list ACCEPTED (also with CRLF, tilde fences, `-` and + bullets); `[ordering]` with no block, `[ ordering ]`, `[Ordering]`, `[ordering] sort these`, `### [extra]` and `##[extra]` REJECTED; ##<TAB>`[extra]` ACCEPTED. Valid corpus bodies brazil-timeline, fibonacci, extra-text, list-markers ACCEPTED. (docs/question-types/ordering.md:141-148)

✓ **Inconsistencies:** The epilogue and section order rule (sections after the epilogue, prose after a section is parse-error) matches question-base.md#epilogue and has invalid examples for both prose between sections and an epilogue after a section. (docs/question-types/ordering.md:175-182, docs/question-types/question-base.md:264-268, examples/invalid/ordering-prose-between-sections.mdq.md, examples/invalid/ordering-epilogue-after-section.mdq.md)

✓ **Omissions:** Both complete Markdown examples in ordering.md validate with no error or warning in mdq-py. (docs/question-types/ordering.md:16-29, docs/question-types/ordering.md:34-89)

✓ **Schema mismatches:** schema/ordering.yaml agrees with the spec on line shape ([level >= 0, text], exactly 2 items), `lines` minItems 2, unique `normalizations`, the `indentation` and `unmatched` enums and defaults, and the descriptions of `lenient` implying `dedent`. (schema/ordering.yaml:17-108, schema/ordering.yaml:138-156)

✓ **Ambiguities:** Indentation unit inference (GCD of indented lines across all blocks, tab = 4 spaces, default 4) is precise and exercised by examples for GCD, tabs, cross-section units and lenient alignment. (docs/question-types/ordering.md:239-250, examples/valid/ordering/indentation-gcd.mdq.md, examples/valid/ordering/tab-indentation.mdq.md, examples/valid/ordering/indentation-unit-across-sections.mdq.md)

✓ **Contradictions:** Grading, automation and pending status agree across ordering.md, question-base.md#automation and responses.md#pending: `unmatched: manual` gives semi-automatic and a pending response with no feedback; binary score with no partial credit. (docs/question-types/ordering.md:302-317, docs/question-types/question-base.md:283-285, docs/responses.md:316-322)

Central risks

• The relations between the answer key, `## [extra]`, `## [accept]` and `## [reject]` (overlap, reachability, distractors in correct answers) are only partly stated, so a valid document can grade one response as both correct and incorrect.

• The ordering grammar is stricter than CommonMark and mdq-py in several places (blank lines, indentation, heading variants), so a second implementation that follows it literally rejects the spec's own full example.

Detailed findings

Contradictions

Severity	Location	Description
HIGH	docs/question-types/ordering.md: 209-210	**F6** A `## [reject]` section may repeat the answer key, which is then both correct and incorrect
HIGH	docs/question-types/ordering.md: 194-198	**F19** Ordering: an accept section may use a distractor that MUST NOT appear in a correct response

F6: A `## [reject]` section may repeat the answer key, which is then both correct and incorrect

Location: docs/question-types/ordering.md:209-210

```
field implies -- and MUST NOT declare an accepted and a rejected section whose
content blocks hold the same lines after that same normalization.
```

Description: The overlap rule (and `accept-reject-overlap` in the table, line 336) only compares `## [accept]` sections with `## [reject]` sections. The `[ordering]` block is also an accepted answer (line 297), but nothing forbids a `## [reject]` whose lines equal it. `mdq validate` accepts `[ordering]\n- a\n- b` followed by `## [reject]\n\n- a\n- b` with no diagnostic.

Impact: Grading line 297-300 says a response matching the `[ordering]` block scores 1 and a response matching any `## [reject]` scores 0: for this document both apply and the spec does not say which wins. Implementations can disagree.

Suggested fix: Extend the MUST NOT to the `[ordering]` block: a `## [reject]` section must not hold the answer key's lines after full normalization (error, `accept-reject-overlap` or a new code). Add an invalid corpus example.

F19: Ordering: an accept section may use a distractor that MUST NOT appear in a correct response

Location: docs/question-types/ordering.md:194-198

```
Its lines are distractors: they are shuffled in among the answer key's lines
and MUST NOT appear in a correct response.
```

Description: The spec says a distractor MUST NOT appear in a correct response, but an `## [accept]` section that contains a line of `## [extra]` is not forbidden: no row of the Additional Rules table covers it, and the reference implementation accepts `## [extra] - c` with `## [accept] - b - c` with no diagnostic. A response that matches that accept section is correct (score 1) and contains a distractor.

Impact: The MUST NOT is not enforceable and contradicts the grading rule for accepted sections. Authors cannot tell if a distractor may be part of an alternative correct answer.

Suggested fix: Decide one: (a) an accept section must not use a line that is only available as a distractor (new error code in the table), or (b) reword line 195 so that a distractor is not part of the answer key, but an accept section may use it.

Ambiguities

Severity	Location	Description
MEDIUM	docs/question-types/ordering.md: 134-136, docs/question-types/ordering.md:106-108	**F8** A declared `content` that disagrees with the body block has no defined meaning
MEDIUM	docs/question-types/ordering.md: 155-157	**F24** Ordering: a fence language inferred as `highlight` triggers `ignored-highlight` under a declared `content: text`
LOW	docs/question-types/ordering.md: 274-276	**F14** Feedback is unspecified when a response matches more than one section
LOW	docs/question-types/ordering.md: 225-229	**F27** Ordering: a distractor that equals a key line after normalization silently becomes a spare copy
LOW	docs/question-types/ordering.md: 292-293	**F28** Ordering: "compared in the order they appear" for repeated lines adds nothing and is unclear

F8: A declared `content` that disagrees with the body block has no defined meaning

Location: docs/question-types/ordering.md:134-136, docs/question-types/ordering.md:106-108

```
markdown `ul` element holding the lines to be ordered. The `content` and
`highlight` frontmatter fields are inferred from this block when they are not
declared, and a declared value wins.
```

Description: `content: code` with a `ul` body, or `content: text` with a fenced code block, is valid (mdq-py: 0K), and the declared value is kept. The spec never says what this changes: whether the `ul` items are then presented as code (raw source in a monospace widget), whether a code block's lines are rendered as Markdown, or whether the line text is extracted differently. For essay `content` describes the student's editor; for ordering it is presented as describing how lines are written, which the body already decides.

Impact: Two hosts render the same document differently; an author cannot predict what `content: text` over a code block does.

Suggested fix: Either make a mismatch an error (`content` must agree with the block type; the field is then only useful in the JSON/YAML form), or define `content` as presentation only (`text` = render each line as inline Markdown, `code` = verbatim monospace) independent of the block. Add a corpus example for the chosen rule.

F24: Ordering: a fence language inferred as `highlight` triggers `ignored-highlight` under a declared `content: text`

Location: docs/question-types/ordering.md:155-157

```
If the block following the `[ordering]` tag is a fenced code block, the
question infers `content: "code"`, and the fence's language, if any, becomes
`highlight`.
```

Description: The spec does not say whether the fence language still becomes `highlight` when the frontmatter declares `content: text`. The reference implementation infers it and then reports `ignored-highlight` (warning, 'should be omitted unless `content` is "code"') for a `highlight` the author never wrote. Tested with `content: text` over a ```` ```python ```` fence.

Impact: Authors get a warning about a field they did not write, which they can only silence by removing the fence language.

Suggested fix: State that `highlight` is inferred from the fence only when `content` is "code" (declared or inferred), or that `ignored-highlight` applies only to a declared `highlight`.

F14: Feedback is unspecified when a response matches more than one section

Location: docs/question-types/ordering.md:274-276

```
A response that matches an accepted or a rejected answer shows that section's
feedback, if it declares one. The `[ordering]` block itself carries no
```

Description: Two `## [accept]` sections with the same normalized lines, or an `## [accept]` equal to the `[ordering]` block, are only warnings (`duplicate-alternative-lines`, `accept-repeats-answer-key`), so valid documents exist where one response matches several sections with different feedback. The spec does not say which feedback is shown (first in document order, all of them, the answer key's generic message).

Impact: Different hosts show different feedback for the same response.

Suggested fix: Say that the first matching answer in document order wins (answer key first, then sections), or that all matching feedback is shown.

F27: Ordering: a distractor that equals a key line after normalization silently becomes a spare copy

Location: docs/question-types/ordering.md:225-229

```
lines, and a distractor may differ from a line of the answer key by its
  level alone.
```

Description: Under `lenient` (or with `dedent`), a distractor that differs from a key line only by level, or a blank distractor under `skip-blanks`, is equal to a key line after normalization. It is then a usable copy, not a distractor. The spec does not say this and no lint reports it; the reference implementation accepts it without a diagnostic.

Impact: An author who intended a distractor gets a question where it is not one.

Suggested fix: Say that distractors are compared after normalization, and add a warning when a distractor is equal to a key line after normalization.

F28: Ordering: "compared in the order they appear" for repeated lines adds nothing and is unclear

Location: docs/question-types/ordering.md:292-293

```
Repeated lines are compared in the order they appear, so a question with
duplicates accepts exactly as many copies as it declares.
```

Description: Grading compares sequences, so repeated lines need no special order rule. The sentence suggests some copy-by-copy pairing. Also, under `skip-blanks`, blank copies are not counted, so the 'exactly as many copies' claim is not true for blank lines.

Impact: Readers may look for a matching algorithm that does not exist.

Suggested fix: State that a response matches an answer key when the two sequences are equal after normalization, and drop the order sentence.

Omissions

Severity	Location	Description
MEDIUM	docs/question-types/ordering.md:159-162, docs/question-types/ordering.md:239-244	**F7** Nested `ul` items: how a text line gets an indentation level is not specified
MEDIUM	docs/responses.md:267-276	**F9** Which responses are malformed versus wrong is not defined for ordering
MEDIUM	docs/question-types/ordering.md:230-234	**F10** Initial indentation shown to the student under `lenient` and `strict` is not specified

Severity	Location	Description
MEDIUM	docs/question-types/ordering.md: 203-210	**F11** Alternatives may use lines the student cannot produce, with no diagnostic
LOW	docs/question-types/ordering.md: 146	**F12** An empty `## [extra]` section is a parse-error with no rule, unlike essay's empty section
LOW	docs/question-types/ordering.md: 346	**F15** `accept-repeats-answer-key` is not described in the prose, and its normalization is not stated
LOW	docs/question-types/ordering.md: 239-250, docs/question-types/ordering.md:261	**F16** Trailing and whitespace-only lines: what the line text keeps is not specified
LOW	docs/question-types/ordering.md: 341-343	**F30** Ordering Additional Rules: no row for an unknown `unmatched` value
LOW	examples/valid/exam/with-ordering.mdq.md:10-16	**F34** Ordering corpus: no example of sections inside an exam, nested lists or content overrides

F7: Nested `ul` items: how a text line gets an indentation level is not specified

Location: docs/question-types/ordering.md:159-162, docs/question-types/ordering.md:239-244

```
An item is exactly one line: an item with a continuation line or a second
paragraph is an error (`parse-error`), whatever the rendered markdown shows.
```

Description: The Indentation section measures the leading whitespace of each line, which is clear for a fenced code block. For a `ul` list the spec never says which whitespace counts: the list marker's indentation, a nested sub-list, or the content offset. A nested list (`- a\n  - b\n- c`) is neither a continuation line nor a second paragraph, so line 161 does not cover it. mdq-py turns b into a separate line at level 1 (unit 2); no corpus example has an indented `ul` item.

Impact: Text questions with indentation (outlines, nested steps) have no defined parse; a second implementation may reject nested lists or flatten them.

Suggested fix: Decide: either nested list items are separate lines whose level comes from nesting depth (or marker column), or any nested list is a `parse-error`. State it in Body and Indentation and add valid/invalid corpus examples.

F9: Which responses are malformed versus wrong is not defined for ordering

Location: docs/responses.md:267-276

```
carry none, so they are compared by content, and a line the question repeats is
matched as many times as it appears. A response may hold a distractor from the
question's `extra` lines -- that is a wrong answer, not a malformed response.
```

Description: Only distractors are classified. The spec does not say whether these are `ResponseError` or just unmatched: a line whose text is neither in `lines` nor in `extra`; more copies of a line than the question presents; a level different from the authored one under `indentation: fixed` (the student cannot change it); an empty list; a response that leaves out some lines. The reject example at ordering.md:82-88 omits the aux lines, which implies a response may be a subset of the pool, but no text states it.

Impact: Other response types reject unknown ids as malformed (responses.md:23-27); for ordering, a response the UI could never produce may be silently graded or sent to manual grading, depending on the implementation.

Suggested fix: Add a rule: a response is a sub-multiset of the presented lines (key plus extra), under `fixed` each level equals the authored level, otherwise `ResponseError`. State that omitting lines is allowed (Parsons-style) or not. Add the cases to the Errors table.

F10: Initial indentation shown to the student under `lenient` and `strict` is not specified

Location: docs/question-types/ordering.md:230-234

```
* `"lenient"`: the student may change indentation freely, but it is ignored
  when grading. This implies the `dedent` normalization, whether or not
  `normalizations` lists it.
* `"strict"`: the student may change indentation, and it is compared like the
  rest of the line.
```

Description: Only `fixed` says lines are presented with their authored indentation. Under `strict` the spec does not say whether the shuffled lines start at their authored level (which gives away part of the answer the student is graded on) or flattened at level 0. Duplicates are defined as same text and same level (line 170), so whether two lines that differ only by level are shown as identical copies under `strict` also depends on this.

Impact: Hosts present the same `strict` question with different difficulty.

Suggested fix: State that under `lenient` and `strict` the lines are presented at level 0 (recommended for `strict`, since indentation is part of the answer), or at the authored level, and add it to the Indentation section.

F11: Alternatives may use lines the student cannot produce, with no diagnostic

Location: docs/question-types/ordering.md:203-210

```
The accept/reject blocks start with a `## [accept]` or `## [reject]` header,
followed by optional observations and the content block. As before, the content
block must be of the same type as the main `[ordering]` block, but repeated
```

Description: The student can only build a response from the presented lines (`[ordering]` plus `## [extra]`). An `## [accept]` or `## [reject]` section that holds a line in neither block, or more copies of a line than the pool offers, can never match. `mdq validate` accepts `[ordering]\n- a\n- b` with `## [accept]\n\n- z\n- b` silently. The `full-metadata` example comment says the distractor is declared in `## [extra]` "so the student can actually build it", so the intent exists but no rule states it.

Impact: A typo in an alternative silently disables its feedback or its credit; the author gets no warning.

Suggested fix: Add a rule (warning, e.g. `unreachable-alternative`): every line of an alternative, after normalization, must be available in the presented pool with enough copies. Add valid/invalid corpus examples.

F12: An empty `## [extra]` section is a parse-error with no rule, unlike essay's empty section

Location: docs/question-types/ordering.md:146

```
ordering_extra  : "##" ws "[" "extra" "]" nl+ content
```

Description: The grammar requires a content block after `## [extra]`, so an empty section fails to parse; mdq-py reports an `[ordering] block must be followed by a code block or a list`, which points at the wrong block. The prose and the Additional Rules table do not list this case. Essay decided that an empty `## [answer-key]` is the same as no section (essay.md:138).

Impact: The behaviour of empty type sections differs between the two types that have them, and the error is not documented.

Suggested fix: Decide whether an empty `## [extra]`/`## [accept]`/`## [reject]` is absent (as essay) or a `parse-error`; add the rule to the table and an example.

F15: `accept-repeats-answer-key` is not described in the prose, and its normalization is not stated

Location: docs/question-types/ordering.md:346

```
| accept                | warning  | accept-repeats-answer-key    | should not repeat the
`[ordering]` block's own lines              |
```

Description: The prose at lines 206-210 covers two sections that repeat each other but never an `## [accept]` that repeats the `[ordering]` block. The table row does not say the comparison uses full normalization, although the corpus does (`accept-repeats-after-dedent`, `accept-repeats-after-skip-blanks`).

Impact: A reader of the prose cannot find the rule, and the table row reads as a raw comparison.

Suggested fix: Add the SHOULD NOT to the Accepted/rejected answers prose and the [^9] footnote to the table row.

F16: Trailing and whitespace-only lines: what the line text keeps is not specified

Location: docs/question-types/ordering.md:239-250, docs/question-types/ordering.md:261

```
considers all lines in the `[ordering]`, `## [extra]`, `## [accept]` and
`## [reject]` blocks, normalizes a tabulation as 4 spaces, and takes the
greatest common divisor of the leading whitespace of every non-blank line that
```

Description: The spec defines how leading whitespace becomes a level but not whether trailing whitespace is part of the text, nor the level of a whitespace-only line. mdq-py keeps trailing spaces (a and a are different lines) and gives level 0 with empty text. The tab rule says 'a tabulation as 4 spaces' without saying whether that is a fixed width or a CommonMark tab stop (`\t` = 4 or 6 columns).

Impact: An alternative copied with invisible trailing spaces silently never matches; implementations can split on these edge cases.

Suggested fix: State that trailing whitespace is stripped (or kept) and that blank lines are level 0 with empty text; say whether tabs expand to a fixed 4 columns or to the next tab stop.

F30: Ordering Additional Rules: no row for an unknown `unmatched` value

Location: docs/question-types/ordering.md:341-343

Mechanical fix

```
| indentation           | error    | schema-error                | must be a known indentation
 mode (schema)                          |
```

Description: The table lists schema-error rows for `content`, `indentation` and `normalizations`, but not for `unmatched`, which the schema also restricts to an enum (schema/ordering.yaml:87-91).

Impact: Incomplete rules table.

Suggested fix: Add `| unmatched | error | schema-error | must be a known unmatched mode (schema) |`.

F34: Ordering corpus: no example of sections inside an exam, nested lists or content overrides

Location: examples/valid/exam/with-ordering.mdq.md:10-16

```
[ordering]
```

Description: The only exam examples with ordering (`with-ordering.mdq.md`, `all-question-types.mdq.md`) have no `## [extra]`, `## [accept]` or `## [reject]` sections and no epilogue, so the exam rule against `---` after the body (docs/exam.md:162-169) is not exercised for ordering. The corpus also has no example of a nested `ul`, a declared `content` that disagrees with the body, a reject equal to the answer key, an accept using a distractor, or an ordering response in a response example.

Impact: The edge cases above are untested in both implementations.

Suggested fix: Add exam and question examples for these cases once their rules are decided.

Inconsistencies

Severity	Location	Description
LOW	docs/question-types/ordering.md:263-265, docs/question-types/ordering.md:302	**F17** Reject sections are called answer keys
LOW	docs/responses.md:267	**F29** responses.md: ordering is not the only response without ids
LOW	docs/question-types/ordering.md:347-350	**F31** Ordering: several table rules have no prose in the document

F17: Reject sections are called answer keys

Location: docs/question-types/ordering.md:263-265, docs/question-types/ordering.md:302

Mechanical fix

```
Normalizations apply to BOTH sides of the comparison: the student's response
and every answer key (the `[ordering]` block and every `## [accept]` and
`## [reject]` section alike). They never alter what is presented to the
```

Description: GLOSSARY.md defines an answer key as the correct answer an instructor declared. Ordering calls the `## [reject]` sections answer keys too, and line 302 'If no answer key matches' depends on that wider sense. Line 7 uses the narrow sense ('the block is the answer key').

Impact: Minor terminology drift.

Suggested fix: Use 'answer key' only for `[ordering]` and `## [accept]`, and a separate term (e.g. 'declared answers' or 'sections') when rejects are included.

F29: responses.md: ordering is not the only response without ids

Location: docs/responses.md:267

Mechanical fix

```
Ordering is the one question type whose response names nothing by id: its lines
```

Description: The table at docs/responses.md:32-40 shows that numeric, short-answer and essay responses also carry no id.

Impact: Misleading statement.

Suggested fix: Reword: 'Ordering lines carry no id, so they are compared by content'.

F31: Ordering: several table rules have no prose in the document

Location: docs/question-types/ordering.md:347-350

```
| reject                | info     | reject-without-feedback      | declares no feedback, which
 is its whole purpose                  |
```

Description: `reject-without-feedback`, `blank-ordering-line`, `unknown-highlight-language` and `redundant-strict-indentation` appear only in the table (footnotes aside); the prose has no SHOULD/MAY that states them. (`accept-repeats-answer-key` is the same case, recorded separately.)

Impact: Readers of the prose miss these rules.

Suggested fix: Add one sentence per rule in the matching section (Accepted/rejected answers, Normalizations, Indentation, Frontmatter).

Grammar violations

Severity	Location	Description
HIGH	docs/question-types/ordering.md:141	**F1** Ordering grammar rejects blank lines between sections, including the spec's own full example
MEDIUM	docs/question-types/ordering.md:141	**F2** Ordering tag rule has no ws? around [ordering], unlike other tags and the CommonMark paragraph
MEDIUM	docs/question-types/ordering.md:141	**F3** Ordering grammar rejects a blank line between [ordering] and its content block
MEDIUM	docs/question-types/ordering.md:146-148, docs/question-types/essay.md:99-100	**F4** Ordering section headings reject CommonMark ATX variants and whitespace-only blank lines
LOW	docs/question-types/ordering.md:142-144	**F5** Ordering grammar accepts mixed content types and two [extra] sections

F1: Ordering grammar rejects blank lines between sections, including the spec's own full example

Location: docs/question-types/ordering.md:141

Mechanical fix

```
ordering : "[" "ordering" "]" nl content epilogue? section*
```

Description: `section*` puts each section heading right after the content block of the previous section. No `blank_line` is allowed between them, and the content block (a CommonMark fenced code block or list) does not include the blank lines that follow it. Tested with Lark (Earley) and stubs for `md_code_block`, `md_ul` and `epilogue`: the body of the full example at docs/question-types/ordering.md:45-88 is REJECTED at the blank line 60, before `## [accept]`. The minimal input `[ordering]` / fenced block / `## [extra]` / fenced block / blank line / `## [accept]` / fenced block is also rejected. The blank line between the main content block and the first section is accepted only if the `epilogue` stub absorbs blank lines, which makes an epilogue of only blank lines. mdq-py accepts all these inputs.

Impact: The grammar contradicts the normal layout of every multi-section ordering question, including the one the spec shows. A reader or implementer who follows the grammar literally rejects valid documents.

Suggested fix: Allow blank lines before each section, for example `ordering : "[" "ordering" "]" nl content epilogue? (blank_line* section)*`. With this change the full example parses, and prose after a section is still rejected.

F2: Ordering tag rule has no ws? around [ordering], unlike other tags and the CommonMark paragraph

Location: docs/question-types/ordering.md:141

```
ordering : "[" "ordering" "]" nl content epilogue? section*
```

Description: The tag line has no ws? before [or after]. The grammar REJECTS [ordering] (up to three spaces of indentation, which CommonMark allows for a paragraph) and [ordering] (trailing spaces, removed from a paragraph by CommonMark). The other tags allow both: essay essay : ws? "[essay]" ws?, numeric answer : ws? "[" "numeric" ..., short-answer accept_block : ws? "[" "short-answer" .... docs/references/grammar.md:59-69 defines the ws? at the start of a block as the CommonMark indentation, so its absence here is a deliberate-looking restriction that the prose does not state. mdq-py accepts both inputs.

Impact: Readers cannot tell whether indentation or trailing spaces on the tag line are an error. The grammar and the reference implementation disagree.

Suggested fix: Write ws? "[" "ordering" "]" ws? nl, as the essay tag does.

F3: Ordering grammar rejects a blank line between [ordering] and its content block

Location: docs/question-types/ordering.md:141

```
ordering : "[" "ordering" "]" nl content epilogue? section*
```

Description: After the tag there is exactly one nl, so the input [ordering] / blank line / fenced code block (or ul list) is REJECTED. The section rules of the same snippet use nl+ after the heading (docs/question-types/ordering.md:146-148), so a blank line is allowed there and not after the tag. The prose ("an [ordering] tag followed either by a code block or a markdown ul element") does not forbid the blank line, and mdq-py accepts it.

Impact: Same snippet, two different rules for the same layout. An implementer who follows the grammar rejects documents that mdq-py accepts.

Suggested fix: Decide whether a blank line may separate the tag from its block. If yes, use blank_line* after the tag line; if no, state it in prose and add an invalid example.

F4: Ordering section headings reject CommonMark ATX variants and whitespace-only blank lines

Location: docs/question-types/ordering.md:146-148, docs/question-types/essay.md:99-100

```
ordering_extra  : "##" ws "[" "extra" "]" nl+ content
ordering_accept : "##" ws "[" "accept" "]" nl+ (observations blank_line*)? content
ordering_reject : "##" ws "[" "reject" "]" nl+ (observations blank_line*)? content
```

Description: The heading rules have no ws? before ## or after], no closing sequence, and use nl+ instead of blank_line*. The grammar REJECTS: ## [extra] (indentation of up to three spaces), ## [extra] (trailing spaces), ## [extra] ## (ATX closing sequence), and ## [extra] followed by a line of only spaces before the content block. CommonMark reads all of them as the same H2 with text [extra], and mdq-py accepts all four. The essay section uses answer_key : ws? "##" ws "[answer-key]" ws? nl md_block* and also admits a setext H2 (docs/question-types/essay.md:92-105); ordering says nothing about setext. Inside the same rule, nl+ before the observations and blank_line* after them treat a whitespace-only line differently.

Impact: The grammar is stricter than both CommonMark and the reference implementation, and inconsistent with the essay section heading.

Suggested fix: Write the heading as in essay, for example ws? "##" ws "[" "extra" "]" ws? nl blank_line*, and state whether a closing sequence is accepted. Apply the essay decision on setext to every type section (a setext H2 [extra] + --- is the same section, with the setext-heading lint), and write it once in question-base.md, since docs/exam.md already applies the setext rule to ordering sections.

F5: Ordering grammar accepts mixed content types and two [extra] sections

Location: docs/question-types/ordering.md:142-144

```
content  : md_code_block | md_ul

section : ordering_extra | ordering_accept | ordering_reject
```

Description: The grammar ACCEPTS `[ordering]` with a fenced code block followed by `## [extra]` with a `ul` list, and ACCEPTS two `## [extra]` sections. The Additional Rules table (docs/question-types/ordering.md:337-338) makes both a `parse-error`. The prose does not say that these two rules are left out of the grammar on purpose, so a reader can take the grammar as the whole syntax.

Impact: Two parse-error rules exist only in the table. Low impact because the table is explicit.

Suggested fix: Either split the rule by content type (`code_body | ul_body`, each with its own section rules) and limit `ordering_extra` to one occurrence, or add a sentence after the snippet that these two rules are checked after parsing.

Schema mismatches

Severity	Location	Description
MEDIUM	schema/ordering.yaml:51-60	**F32** schema/ordering.yaml: `content` default "text" is not in the spec
LOW	schema/ordering.yaml:62-68	**F33** schema/ordering.yaml: `highlight` minLength 1 is not in the spec

F32: schema/ordering.yaml: `content` default "text" is not in the spec

Location: schema/ordering.yaml:51-60

```
        default: text
```

Description: ordering.md only says `content` is inferred from the body. A YAML/JSON document has no body to infer from, and the spec never states the value of an omitted `content` there. The schema declares `default: text`, which also disagrees with the inference for a fenced body. essay.md, in contrast, states 'When omitted, it is "text"'.

Impact: A YAML document with code lines and no `content` is read as text by one implementation and possibly rejected or inferred differently by another.

Suggested fix: State in ordering.md that `content` defaults to "text" in the YAML/JSON form (and that the Markdown form infers it), or drop the schema default.

F33: schema/ordering.yaml: `highlight` minLength 1 is not in the spec

Location: schema/ordering.yaml:62-68

Mechanical fix

```
        minLength: 1
```

Description: The schema rejects `highlight: ""`, but ordering.md does not say that an empty highlight is invalid and the Additional Rules table has no schema-error row for `highlight`.

Impact: Minor: an undocumented error.

Suggested fix: Add `| highlight | error | schema-error | must not be empty (schema) |` to the table.

Prioritized recommendations

P1: Write the consistency rules between key, distractors and alternatives, and the malformed-response rules, so no valid document grades a response two ways. (Findings: F6, F19, F11, F27, F15, F9, F14, F28)

P1: Apply the mechanical fixes: blank lines between sections in the grammar, the 'answer key' term, the responses.md id sentence, and the missing schema-error rows. (Findings: F1, F17, F29, F30, F33)

P2: Align the ordering grammar with CommonMark and essay, and define line text and indentation for list items and whitespace. (Findings: F2, F3, F4, F5, F12, F7, F16, F10)

P2: Settle the meaning of `content` and `highlight` for ordering, shared with essay. (Findings: F8, F24, F32)

P3: Complete the prose for table-only rules and the corpus for exams with ordering sections. (Findings: F31, F34)

GitHub issues

Each block below is a complete issue, ready to paste.

````
--- ISSUE 1 ---
# [Spec] Ordering: consistency rules between the answer key, distractors and alternatives

**Labels:** high, medium, low

## Problem

Define, after full normalization, how `[ordering]`, `## [extra]`, `## [accept]` and `##
[reject]` may relate. Recommended: error when a reject equals the key (extend `accept-reject-
overlap`); error when an accept uses a line only available as a distractor; warning when an
alternative uses a line or more copies than the presented pool has; warning when a distractor
equals a key line after normalization; state `accept-repeats-answer-key` in prose.

## Evidence

- `docs/question-types/ordering.md:209-210`: A `## [reject]` section may repeat the answer key,
 which is then both correct and incorrect

  ```markdown
  field implies -- and MUST NOT declare an accepted and a rejected section whose
  content blocks hold the same lines after that same normalization.
  ```
- `docs/question-types/ordering.md:194-198`: Ordering: an accept section may use a distractor
that MUST NOT appear in a correct response

  ```markdown
  Its lines are distractors: they are shuffled in among the answer key's lines
  and MUST NOT appear in a correct response.
  ```
- `docs/question-types/ordering.md:203-210`: Alternatives may use lines the student cannot
produce, with no diagnostic

  ```markdown
  The accept/reject blocks start with a `## [accept]` or `## [reject]` header,
  followed by optional observations and the content block. As before, the content
  block must be of the same type as the main `[ordering]` block, but repeated
  ```
- `docs/question-types/ordering.md:225-229`: Ordering: a distractor that equals a key line
after normalization silently becomes a spare copy

  ```markdown
  lines, and a distractor may differ from a line of the answer key by its
    level alone.
  ```
- `docs/question-types/ordering.md:346`: `accept-repeats-answer-key` is not described in the
prose, and its normalization is not stated

  ```markdown
  | accept                | warning  | accept-repeats-answer-key    | should not repeat the
`[ordering]` block's own lines              |
  ```

## Impact

- **A `## [reject]` section may repeat the answer key, which is then both correct and
incorrect:** Grading line 297-300 says a response matching the `[ordering]` block scores 1 and
a response matching any `## [reject]` scores 0: for this document both apply and the spec does
not say which wins. Implementations can disagree.
- **Ordering: an accept section may use a distractor that MUST NOT appear in a correct
response:** The MUST NOT is not enforceable and contradicts the grading rule for accepted
sections. Authors cannot tell if a distractor may be part of an alternative correct answer.
- **Alternatives may use lines the student cannot produce, with no diagnostic:** A typo in an
alternative silently disables its feedback or its credit; the author gets no warning.
- **Ordering: a distractor that equals a key line after normalization silently becomes a spare
copy:** An author who intended a distractor gets a question where it is not one.
- **`accept-repeats-answer-key` is not described in the prose, and its normalization is not
stated:** A reader of the prose cannot find the rule, and the table row reads as a raw
comparison.

## Suggested fix
````

```
- **A `## [reject]` section may repeat the answer key, which is then both correct and
incorrect:** Extend the MUST NOT to the `[ordering]` block: a `## [reject]` section must not
hold the answer key's lines after full normalization (error, `accept-reject-overlap` or a new
code). Add an invalid corpus example.
- **Ordering: an accept section may use a distractor that MUST NOT appear in a correct
response:** Decide one: (a) an accept section must not use a line that is only available as a
distractor (new error code in the table), or (b) reword line 195 so that a distractor is not
part of the answer key, but an accept section may use it.
- **Alternatives may use lines the student cannot produce, with no diagnostic:** Add a rule
(warning, e.g. `unreachable-alternative`): every line of an alternative, after normalization,
must be available in the presented pool with enough copies. Add valid/invalid corpus examples.
- **Ordering: a distractor that equals a key line after normalization silently becomes a spare
copy:** Say that distractors are compared after normalization, and add a warning when a
distractor is equal to a key line after normalization.
- **`accept-repeats-answer-key` is not described in the prose, and its normalization is not
stated:** Add the SHOULD NOT to the Accepted/rejected answers prose and the [^9] footnote to
the table row.

## Acceptance criteria

- [ ] ordering.md states each relation and the table has a row per rule
- [ ] Each rule has a valid or invalid corpus example
- [ ] mdq-py reports each case

--- END ISSUE 1 ---
```

````
--- ISSUE 2 ---
# [Spec] Ordering: define malformed responses, matching and feedback precedence

**Labels:** medium, low

## Problem

State which ordering responses are `ResponseError` (line not in the pool, too many copies,
changed level under `fixed`) and whether a response may leave lines out. Define matching as
equality of the normalized sequences and say which section's feedback wins when several match
(recommended: first in document order, answer key first).

## Evidence

- `docs/responses.md:267-276`: Which responses are malformed versus wrong is not defined for
ordering

  ```markdown
  carry none, so they are compared by content, and a line the question repeats is
  matched as many times as it appears. A response may hold a distractor from the
  question's `extra` lines -- that is a wrong answer, not a malformed response.
  ```
- `docs/question-types/ordering.md:274-276`: Feedback is unspecified when a response matches
more than one section

  ```markdown
  A response that matches an accepted or a rejected answer shows that section's
  feedback, if it declares one. The `[ordering]` block itself carries no
  ```
- `docs/question-types/ordering.md:292-293`: Ordering: "compared in the order they appear" for
repeated lines adds nothing and is unclear

  ```markdown
  Repeated lines are compared in the order they appear, so a question with
  duplicates accepts exactly as many copies as it declares.
  ```

## Impact

- **Which responses are malformed versus wrong is not defined for ordering:** Other response
types reject unknown ids as malformed (responses.md:23-27); for ordering, a response the UI
could never produce may be silently graded or sent to manual grading, depending on the
implementation.
- **Feedback is unspecified when a response matches more than one section:** Different hosts
show different feedback for the same response.
- **Ordering: "compared in the order they appear" for repeated lines adds nothing and is
unclear:** Readers may look for a matching algorithm that does not exist.

## Suggested fix

- **Which responses are malformed versus wrong is not defined for ordering:** Add a rule: a
response is a sub-multiset of the presented lines (key plus extra), under `fixed` each level
equals the authored level, otherwise `ResponseError`. State that omitting lines is allowed
(Parsons-style) or not. Add the cases to the Errors table.
- **Feedback is unspecified when a response matches more than one section:** Say that the first
 matching answer in document order wins (answer key first, then sections), or that all matching
 feedback is shown.
- **Ordering: "compared in the order they appear" for repeated lines adds nothing and is
unclear:** State that a response matches an answer key when the two sequences are equal after
normalization, and drop the order sentence.

## Acceptance criteria

- [ ] responses.md lists the malformed ordering cases
- [ ] ordering.md defines matching without the 'order they appear' sentence
- [ ] Feedback precedence is stated

--- END ISSUE 2 ---
````

````
--- ISSUE 3 ---
# [Spec] Ordering grammar: tag line, blank lines and section headings

**Labels:** medium, low

## Problem

Align the ordering grammar with CommonMark and with essay: `ws?` around `[ordering]`, optional
blank line after the tag, ATX heading variants, setext H2 as in essay (written once in
question-base.md), and a decision for empty sections (absent, as essay, or parse-error). Either
 encode the same-block-type and single-extra rules in the grammar or say they are checked after
 parsing.

## Evidence

- `docs/question-types/ordering.md:141`: Ordering tag rule has no ws? around [ordering], unlike
 other tags and the CommonMark paragraph

  ```lark
  ordering : "[" "ordering" "]" nl content epilogue? section*
  ```
- `docs/question-types/ordering.md:141`: Ordering grammar rejects a blank line between
[ordering] and its content block

  ```lark
  ordering : "[" "ordering" "]" nl content epilogue? section*
  ```
- `docs/question-types/ordering.md:146-148, docs/question-types/essay.md:99-100`: Ordering
section headings reject CommonMark ATX variants and whitespace-only blank lines

  ```lark
  ordering_extra  : "##" ws "[" "extra" "]" nl+ content
  ordering_accept : "##" ws "[" "accept" "]" nl+ (observations blank_line*)? content
  ordering_reject : "##" ws "[" "reject" "]" nl+ (observations blank_line*)? content
  ```
- `docs/question-types/ordering.md:142-144`: Ordering grammar accepts mixed content types and
two [extra] sections

  ```lark
  content  : md_code_block | md_ul

  section : ordering_extra | ordering_accept | ordering_reject
  ```
- `docs/question-types/ordering.md:146`: An empty `## [extra]` section is a parse-error with no
 rule, unlike essay's empty section

  ```lark
  ordering_extra  : "##" ws "[" "extra" "]" nl+ content
  ```

## Impact

- **Ordering tag rule has no ws? around [ordering], unlike other tags and the CommonMark
paragraph:** Readers cannot tell whether indentation or trailing spaces on the tag line are an
error. The grammar and the reference implementation disagree.
- **Ordering grammar rejects a blank line between [ordering] and its content block:** Same
snippet, two different rules for the same layout. An implementer who follows the grammar
rejects documents that mdq-py accepts.
- **Ordering section headings reject CommonMark ATX variants and whitespace-only blank lines:**
 The grammar is stricter than both CommonMark and the reference implementation, and
inconsistent with the essay section heading.
- **Ordering grammar accepts mixed content types and two [extra] sections:** Two parse-error
rules exist only in the table. Low impact because the table is explicit.
- **An empty `## [extra]` section is a parse-error with no rule, unlike essay's empty
section:** The behaviour of empty type sections differs between the two types that have them,
and the error is not documented.

## Suggested fix

- **Ordering tag rule has no ws? around [ordering], unlike other tags and the CommonMark
````

```
paragraph:** Write `ws? "[" "ordering" "]" ws? nl`, as the essay tag does.
- **Ordering grammar rejects a blank line between [ordering] and its content block:** Decide
whether a blank line may separate the tag from its block. If yes, use `blank_line*` after the
tag line; if no, state it in prose and add an invalid example.
- **Ordering section headings reject CommonMark ATX variants and whitespace-only blank lines:**
 Write the heading as in essay, for example `ws? "##" ws "[" "extra" "]" ws? nl blank_line*`,
and state whether a closing sequence is accepted. Apply the essay decision on setext to every
type section (a setext H2 `[extra]` + `---` is the same section, with the `setext-heading`
lint), and write it once in question-base.md, since docs/exam.md already applies the setext
rule to ordering sections.
- **Ordering grammar accepts mixed content types and two [extra] sections:** Either split the
rule by content type (`code_body | ul_body`, each with its own section rules) and limit
`ordering_extra` to one occurrence, or add a sentence after the snippet that these two rules
are checked after parsing.
- **An empty `## [extra]` section is a parse-error with no rule, unlike essay's empty
section:** Decide whether an empty `## [extra]`/`## [accept]`/`## [reject]` is absent (as
essay) or a `parse-error`; add the rule to the table and an example.

## Acceptance criteria

- [ ] The grammar accepts every layout mdq-py and CommonMark accept for the chosen rules
- [ ] Empty sections have a table row and an example
- [ ] Lark tests pass for the new snippet

--- END ISSUE 3 ---
```

````
--- ISSUE 4 ---
# [Spec] Ordering: line text and indentation of list items and whitespace

**Labels:** medium, low

## Problem

Define how a `ul` item gets its level (nested lists: depth or marker column, or parse-error),
whether trailing whitespace is part of the text, the level of whitespace-only lines, tab
expansion, and the indentation shown to the student under `lenient` and `strict` (recommended:
level 0 under `strict`).

## Evidence

- `docs/question-types/ordering.md:159-162, docs/question-types/ordering.md:239-244`: Nested
`ul` items: how a text line gets an indentation level is not specified

  ```markdown
  An item is exactly one line: an item with a continuation line or a second
  paragraph is an error (`parse-error`), whatever the rendered markdown shows.
  ```
- `docs/question-types/ordering.md:239-250, docs/question-types/ordering.md:261`: Trailing and
whitespace-only lines: what the line text keeps is not specified

  ```markdown
  considers all lines in the `[ordering]`, `## [extra]`, `## [accept]` and
  `## [reject]` blocks, normalizes a tabulation as 4 spaces, and takes the
  greatest common divisor of the leading whitespace of every non-blank line that
  ```
- `docs/question-types/ordering.md:230-234`: Initial indentation shown to the student under
`lenient` and `strict` is not specified

  ```markdown
  * `"lenient"`: the student may change indentation freely, but it is ignored
    when grading. This implies the `dedent` normalization, whether or not
    `normalizations` lists it.
  * `"strict"`: the student may change indentation, and it is compared like the
    rest of the line.
  ```

## Impact

- **Nested `ul` items: how a text line gets an indentation level is not specified:** Text
questions with indentation (outlines, nested steps) have no defined parse; a second
implementation may reject nested lists or flatten them.
- **Trailing and whitespace-only lines: what the line text keeps is not specified:** An
alternative copied with invisible trailing spaces silently never matches; implementations can
split on these edge cases.
- **Initial indentation shown to the student under `lenient` and `strict` is not specified:**
Hosts present the same `strict` question with different difficulty.

## Suggested fix

- **Nested `ul` items: how a text line gets an indentation level is not specified:** Decide:
either nested list items are separate lines whose level comes from nesting depth (or marker
column), or any nested list is a `parse-error`. State it in Body and Indentation and add
valid/invalid corpus examples.
- **Trailing and whitespace-only lines: what the line text keeps is not specified:** State that
 trailing whitespace is stripped (or kept) and that blank lines are level 0 with empty text;
say whether tabs expand to a fixed 4 columns or to the next tab stop.
- **Initial indentation shown to the student under `lenient` and `strict` is not specified:**
State that under `lenient` and `strict` the lines are presented at level 0 (recommended for
`strict`, since indentation is part of the answer), or at the authored level, and add it to the
 Indentation section.

## Acceptance criteria

- [ ] Body and Indentation sections cover nested lists, whitespace and tabs
- [ ] Initial presentation under each indentation mode is stated
- [ ] Corpus has a nested-list example
````

```
--- END ISSUE 4 ---
```

````
--- ISSUE 5 ---
# [Spec] Ordering: meaning and default of `content` and `highlight`

**Labels:** medium

## Problem

Decide what a declared `content` that disagrees with the body does (recommended: error, since
the block already decides it), whether the fence language is inferred as `highlight` under
`content: text`, and the default of `content` in the YAML/JSON form (align with the schema
`default: text` or drop it).

## Evidence

- `docs/question-types/ordering.md:134-136, docs/question-types/ordering.md:106-108`: A
declared `content` that disagrees with the body block has no defined meaning

  ```markdown
  markdown `ul` element holding the lines to be ordered. The `content` and
  `highlight` frontmatter fields are inferred from this block when they are not
  declared, and a declared value wins.
  ```
- `docs/question-types/ordering.md:155-157`: Ordering: a fence language inferred as `highlight`
 triggers `ignored-highlight` under a declared `content: text`

  ```markdown
  If the block following the `[ordering]` tag is a fenced code block, the
  question infers `content: "code"`, and the fence's language, if any, becomes
  `highlight`.
  ```
- `schema/ordering.yaml:51-60`: schema/ordering.yaml: `content` default "text" is not in the
spec

  ```yaml
          default: text
  ```

## Impact

- **A declared `content` that disagrees with the body block has no defined meaning:** Two hosts
 render the same document differently; an author cannot predict what `content: text` over a
code block does.
- **Ordering: a fence language inferred as `highlight` triggers `ignored-highlight` under a
declared `content: text`:** Authors get a warning about a field they did not write, which they
can only silence by removing the fence language.
- **schema/ordering.yaml: `content` default "text" is not in the spec:** A YAML document with
code lines and no `content` is read as text by one implementation and possibly rejected or
inferred differently by another.

## Suggested fix

- **A declared `content` that disagrees with the body block has no defined meaning:** Either
make a mismatch an error (`content` must agree with the block type; the field is then only
useful in the JSON/YAML form), or define `content` as presentation only (`text` = render each
line as inline Markdown, `code` = verbatim monospace) independent of the block. Add a corpus
example for the chosen rule.
- **Ordering: a fence language inferred as `highlight` triggers `ignored-highlight` under a
declared `content: text`:** State that `highlight` is inferred from the fence only when
`content` is "code" (declared or inferred), or that `ignored-highlight` applies only to a
declared `highlight`.
- **schema/ordering.yaml: `content` default "text" is not in the spec:** State in ordering.md
that `content` defaults to "text" in the YAML/JSON form (and that the Markdown form infers it),
 or drop the schema default.

## Acceptance criteria

- [ ] ordering.md states the rule and the default
- [ ] Schema description and default agree with it
- [ ] Corpus example for the mismatch case
````

````
--- END ISSUE 5 ---

--- ISSUE 6 ---
# [Spec] Ordering: rules table prose and corpus coverage

**Labels:** low

## Problem

Add a prose sentence for each table-only rule (`reject-without-feedback`, `blank-ordering-
line`, `unknown-highlight-language`, `redundant-strict-indentation`) and add exam examples with
 ordering sections and epilogue, plus examples for the cases decided in the other issues.

## Evidence

- `docs/question-types/ordering.md:347-350`: Ordering: several table rules have no prose in the
 document

  ```markdown
  | reject                | info     | reject-without-feedback      | declares no feedback,
which is its whole purpose                 |
  ```
- `examples/valid/exam/with-ordering.mdq.md:10-16`: Ordering corpus: no example of sections
inside an exam, nested lists or content overrides

  ```markdown
  [ordering]
  ```

## Impact

- **Ordering: several table rules have no prose in the document:** Readers of the prose miss
these rules.
- **Ordering corpus: no example of sections inside an exam, nested lists or content
overrides:** The edge cases above are untested in both implementations.

## Suggested fix

- **Ordering: several table rules have no prose in the document:** Add one sentence per rule in
 the matching section (Accepted/rejected answers, Normalizations, Indentation, Frontmatter).
- **Ordering corpus: no example of sections inside an exam, nested lists or content
overrides:** Add exam and question examples for these cases once their rules are decided.

## Acceptance criteria

- [ ] Every table row has a matching sentence in the prose
- [ ] An exam example has an ordering question with sections

--- END ISSUE 6 ---
````

````
--- ISSUE 7 ---
# Mechanical fixes

**Labels:** mechanical, high, low

## Problem

Each fix below follows from its finding and needs no design decision, so they can be applied
together in one pass and reviewed by reading the diff. Severity still applies: a mechanical fix
 can matter a lot.

### F1 (high): Ordering grammar rejects blank lines between sections, including the spec's own
full example

`docs/question-types/ordering.md:141`

```lark
ordering : "[" "ordering" "]" nl content epilogue? section*
```

**Suggested fix:** Allow blank lines before each section, for example `ordering : "["
"ordering" "]" nl content epilogue? (blank_line* section)*`. With this change the full example
parses, and prose after a section is still rejected.

### F17 (low): Reject sections are called answer keys

`docs/question-types/ordering.md:263-265, docs/question-types/ordering.md:302`

```markdown
Normalizations apply to BOTH sides of the comparison: the student's response
and every answer key (the `[ordering]` block and every `## [accept]` and
`## [reject]` section alike). They never alter what is presented to the
```

**Suggested fix:** Use 'answer key' only for `[ordering]` and `## [accept]`, and a separate
term (e.g. 'declared answers' or 'sections') when rejects are included.

### F29 (low): responses.md: ordering is not the only response without ids

`docs/responses.md:267`

```markdown
Ordering is the one question type whose response names nothing by id: its lines
```

**Suggested fix:** Reword: 'Ordering lines carry no id, so they are compared by content'.

### F30 (low): Ordering Additional Rules: no row for an unknown `unmatched` value

`docs/question-types/ordering.md:341-343`

```markdown
| indentation           | error    | schema-error                | must be a known indentation
 mode (schema)                          |
```

**Suggested fix:** Add `| unmatched | error | schema-error | must be a known unmatched mode
(schema) |`.

### F33 (low): schema/ordering.yaml: `highlight` minLength 1 is not in the spec

`schema/ordering.yaml:62-68`

```yaml
        minLength: 1
```

**Suggested fix:** Add `| highlight | error | schema-error | must not be empty (schema) |` to
the table.
````

```
## Acceptance criteria

- [ ] F1: Ordering grammar rejects blank lines between sections, including the spec's own full
example
- [ ] F17: Reject sections are called answer keys
- [ ] F29: responses.md: ordering is not the only response without ids
- [ ] F30: Ordering Additional Rules: no row for an unknown `unmatched` value
- [ ] F33: schema/ordering.yaml: `highlight` minLength 1 is not in the spec

--- END ISSUE 7 ---
```